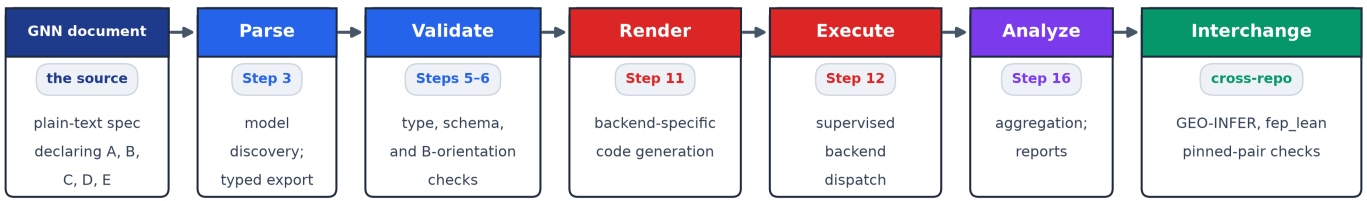
Abstract

Abstract I

Generalized Notation Notation at a Glance

GNN v4.2.1 — one text specification, many traceable realizations



New in validation — the B-tensor orientation check reads every B literal as column-stochastic B[s′, s, a] (the pymdp convention); --transpose-b maps row-stochastic textbook literals onto the canonical order instead of failing them. New in execution — the bnlearn lane generates runnable Python from the same parsed model and skips with a recorded status when the bnlearn runtime is absent.

---

25 pipeline steps (0–24) · 9 model families · 12 registered render backends, 12 render-target adapters at Step 12 · 164 MCP tools

Figure 1: The GeneralizedNotationNotation (GNN) pipeline at a glance, generated from the producer's token map at commit 7d402507e. A plain-text specification — the `StateSpaceBlock` declaring the model's tensors, `A`, `B`, `C`, `D`, and `E` for the discrete categorical kind of eq. 1 through eq. 8 and its own matrix vocabulary for the other model kinds — is parsed (Step 3), type-checked and validated (Steps 5–6),

rendered as backend-specific code (Step 11), executed (Step 12), and analyzed (Step 16), before artifacts flow into the cross-repository interchange checks against the pinned GEO-INFER and fep_lean pairs. Read the cards left to right: each header names the stage, each pill names the pipeline step (or the source role), and the body states what the stage produces. The validation stage includes the B-tensor

orientation check — every discrete `B` literal is read as column-stochastic `B[s', s, a]`, and `--transpose-b` maps row-stochastic textbook literals onto that canonical order — and the execution stage includes the bnlearn lane, which generates runnable Python from the same parsed model and skips with a recorded status when its runtime is absent. The footer states the scale the panel summarizes: 25 pipeline

steps, 9 model families, 12 registered render backends with 12 render targets having Step 12 execution adapters, and 164 Model Context Protocol tools.

Abstract I

Active Inference offers a unifying account of perception, learning, and action under the free energy principle (Friston 2010), yet the generative models at its core are still communicated ad hoc: scattered across prose descriptions, bespoke notebooks, and framework-specific code that rarely agree. This fragmentation makes published models hard to reproduce, compare, or port between tools, and it raises the barrier for newcomers learning the formalism (Parr et al. 2022). GeneralizedNotationNotation (GNN) addresses this gap with a standardized, human- and machine-readable text language for specifying Active Inference generative models, paired with a 25-step processing pipeline that transforms a single specification into validation, visualization, simulation, and analysis artifacts (Smékal and Friedman 2023). A GNN file denotes a generative model, and the language lets that model take the several shapes the field actually uses: the notation

blocks a file declares — categorical `A`/`B`/`C`/`D`[/`E`] tensors, linear-Gaussian system matrices `F`/`H`/`Q`/`R` with a Gaussian prior, per-agent and per-level compositions — fix its model kind, which the pipeline classifies structurally and carries through rendering and execution, recording explicitly which registered backends can render and run a model of each kind and which report it unsupported. GNN's "Triple Play" treats each model as three coordinated views — a textual specification, graphical visualizations, and executable cognitive models — so that outputs share an authored source and can be checked for consistency through identity, parameter, semantic, and numerical receipts. The framework spans 38 exemplar specifications (29 discrete-state and 9 continuous linear-Gaussian) across 9 model families and 12 registered rendering backends, with explicit gates for

cross-format semantic fidelity and cross-framework reliability. GNN 3.0.0 layered safe-by-design long-running orchestration on top of this language and pipeline — durable observation streams, resumable run sessions, and auditable container plans that generate, validate, and replay data only, with no live infrastructure mutation. We describe the language, the model kinds it denotes, the pipeline architecture, and the validation contracts and source-bound checks used to assess specification round trips and admitted executions across simulation backends — with coverage gaps recorded as explicit profiled-unsupported statuses rather than silently omitted — providing inspectable infrastructure for reproducing and comparing Active Inference models within the recorded capability and verification boundaries.